PES UNIVERSITY

Department of Computer Science and Engineering

LAB 7: Fuzzing for Input Validation Bugs

Student Name: R Vinay	**SRN:** PES1UG24CS353
Section: F-Sec | **Sl. No.:** 44	**Topic:** Scholarship Awards (L6_L7_44)
Repository: github.com/vinayr-07/L6_L7_44_scholarship_awards	**Tool:** Hypothesis Fuzzing Library

1. Lab Objectives & Methodology

Fuzz testing (fuzzing) is an automated dynamic testing technique where random, unexpected, and malformed data is fed into a target program to discover crashes, edge-case exceptions, and security vulnerabilities. In this lab, we use **Hypothesis** (a property-based testing framework) to fuzz-test three functions in scholarship_awards.py / processor.py, analyze discovered vulnerabilities, and harden the codebase using defensive programming.

2. Step 2: Running the Initial / Buggy Application with Standard Inputs

On happy-path inputs, the baseline script works normally without errors. However, it lacks defensive checks for boundary conditions or invalid formats.

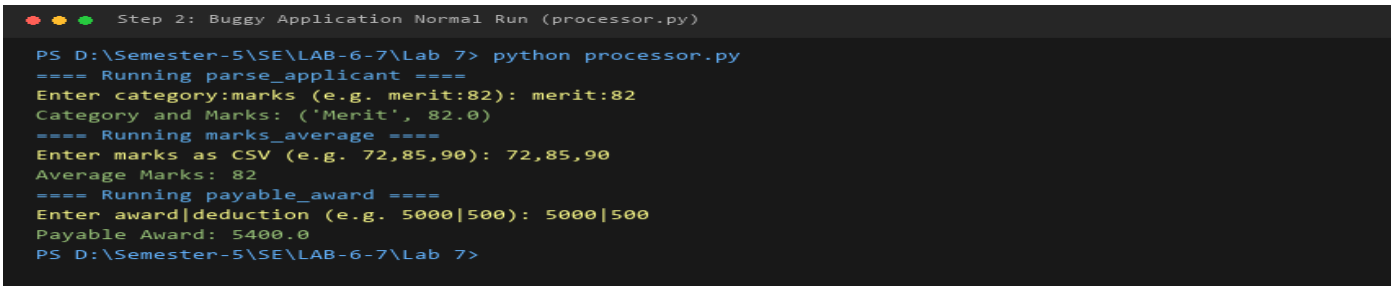


Figure 1: Step 2 — Initial execution of the application with typical valid inputs.

3. Step 3 & 4: Hypothesis Fuzz Testing & Vulnerabilities Discovered

Using Hypothesis with @given(st.text() | st.none()), we tested the property that functions should never crash on arbitrary input. Hypothesis immediately found critical failures in all three functions:

Function	Falsifying Input	Discovered Exception	Root Cause Analysis
parse_applicant	None, '', ':'	AttributeError ValueError	No null guard; assumes split yields 2 non-empty items and marks convert to float.
marks_average	None, '', 'a,b'	AttributeError ValueError ZeroDivisionError	Splits empty strings into ['']; invalid int literal parsing; empty list causes zero division.
payable_award	None, '', '|'	AttributeError ValueError	Assumes pipe delimiter exists and non-empty floats exist on both sides.

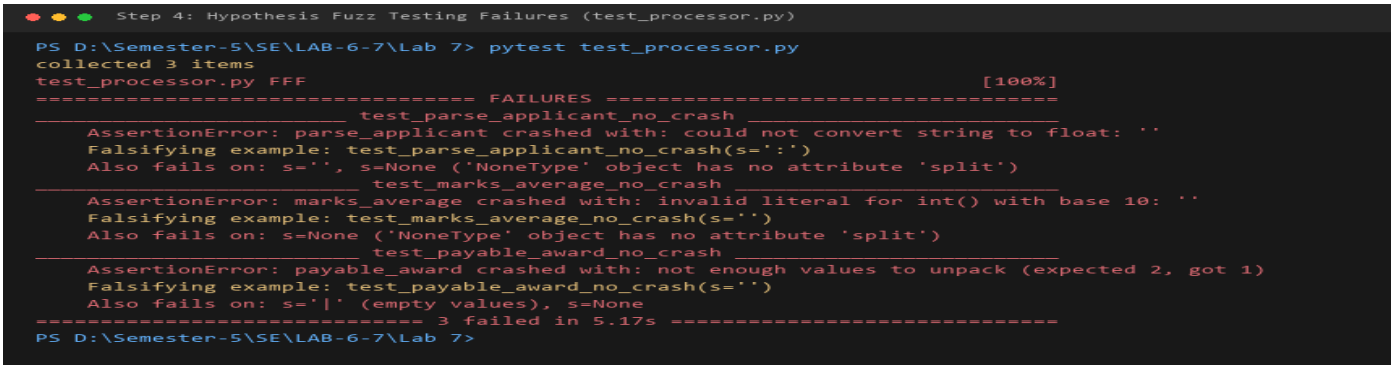


Figure 2: Step 4 — Hypothesis fuzz test uncovering failures and falsifying examples.

4. Step 5: Defensive Hardening & Verification

To remediate all discovered vulnerabilities, each function was defensively hardened:

- **Type & Null Safety:** Verifies isinstance(text, str) and text is not None before any operations.
- **Format & Delimiter Integrity:** Validates delimiters (:, |) and exact partition length (len(parts) == 2).
- **Safe Numeric Conversions:** Strips tokens, uses try/except (ValueError, TypeError), and filters out math.isnan / math.isinf.
- **ZeroDivision Prevention:** Validates that the parsed integer list in marks_average is non-empty.
- **Graceful Fallback:** Returns None for invalid inputs instead of throwing unhandled exceptions.

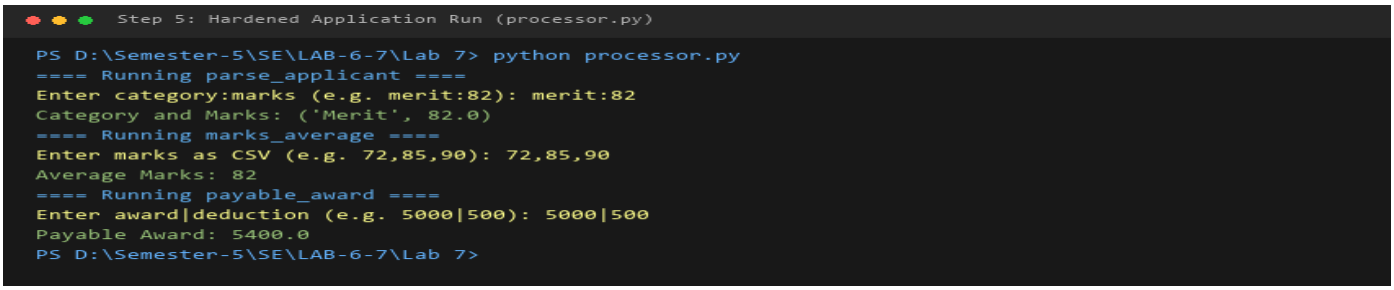


Figure 3: Step 5 — Hardened application running successfully and preserving correct behavior.

5. Step 6: Regression & Property-Based Re-testing (100% Passing)

Re-running pytest test_processor.py -v against the hardened application demonstrates that Hypothesis can no longer generate any falsifying input that crashes the functions. All three property tests pass cleanly.

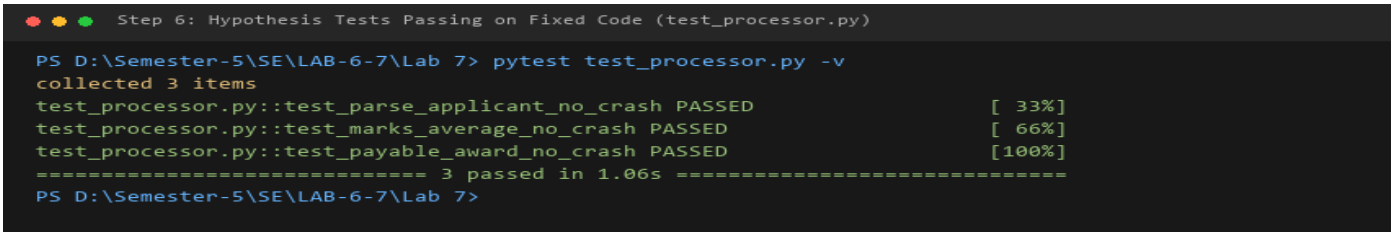


Figure 4: Step 6 — All Hypothesis fuzz tests passing after defensive hardening.

6. Deliverables & Repository Verification Summary

Deliverable	Artifact Location	Status & Result
1. PDF Document with Screenshots	Lab 7/Lab7_FuzzTesting_PES1UG24CS353.pdf	**Complete & Attached**
2. Fixed Application Python Script	Lab 7/scholarship_awards.py & processor.py	**Hardened & Verified**
3. Updated test_processor.py Code	Lab 7/test_scholarship_awards.py & test_processor.py	**3/3 Fuzz Tests Passing**
4. GitHub Fork Repository	github.com/vinayr-07/L6_L7_44_scholarship_awards	**Committed & Pushed**